

CA-LegalGate（律核）系统说明书

****软件名称****: CA-LegalGate（律核）• 三通道异构法律问答路由系统
****版本****: 0.1.0（`lawgate/\_\_init\_\_.py: \_\_version\_\_`）
****文档版本****: v0.1（软件著作权申报用系统说明书）
****运行平台****: Windows 11 / Python 3.13 / CPU-only（可迁移至 CUDA）
****阅读提示****: 本说明书中的每一个类名、函数名、字段名、SQL 表名都取自本仓库的实际代码；
凡涉及****数值结论****之处均给出产物文件路径。系统当前的数据来源为人工录入的关键条文种子语料
与合成案号库，相关限制在 §10 与 `docs/model\_card.md` / `docs/DATA\_GAP.md` 中说明。

目录

- 1. [系统概述] (#1-系统概述)
- 2. [运行环境与部署] (#2-运行环境与部署)
- 3. [模块说明] (#3-模块说明)
- 4. [三个典型使用场景] (#4-三个典型使用场景)
- 5. [数据结构说明] (#5-数据结构说明)
- 6. [API 说明] (#6-api-说明)
- 7. [界面说明] (#7-界面说明)
- 8. [错误处理与降级路径] (#8-错误处理与降级路径)
- 9. [性能与算力管理] (#9-性能与算力管理)
- 10. [已知限制与免责声明] (#10-已知限制与免责声明)

1. 系统概述

1.1 系统定位

CA-LegalGate 是一套****免训练（training-free）****的中文法律问答系统。它不微调任何模型，
而是把“检索增强生成（RAG）”的取舍问题转化为****门控决策问题****：

对每一个用户提问，判断它究竟需要“查库”还是“直接答”，并据此走三条****异构****通道之一；
需要查库时进一步区分“查结构化事实库”还是“查语义向量库”。

系统由此在****不牺牲准确率****的前提下减少检索调用次数，并把每一次决策暴露为可审计的 `trace`。

1.2 三条通道

通道	名称	检索调用	能力来源	实现
***B**	确定性结构化	0	SQLite 事实库（法条 / 案号 / 沿革 / FTS）	`lawgate/channel/b_structured.py: ChannelB.run()`
***A**	直接生成	0	语言模型自身参数知识	`lawgate/channel/llm_base.py: BaseLLM.generate(context=None)`
***C**	语义检索增强	1	向量库（bge 编码 + chroma + BM25 重排）	`lawgate/channel/c_semantic.py: Retriever.retrieve()`

1.3 核心机制

机制	说明	代码位置
意图检测与槽位抽取	纯正则 + 词典，零模型依赖，完全确定性	`lawgate/gate/intent.py: detect_intent()`
槽位继承	只继承 **上一轮经通道 B 事实确认** 的槽位（`topic` / `law_short`），不从未确认轮次抓词	`intent.py` 第 6 步
草稿不确定性信号	取前 `k_draft=20` 个 token 的 top-20 logprob 分布，算 4 个有界信号	`lawgate/gate/signal.py: SIGNALS`
确定性复杂度评分	长度/案例词/主题词/多轮/比较问法 条号原文扣分 + 类别先验	`lawgate/gate/complexity.py`

```
complexity_score() |  
| 桶级阈值 | b1 概念 / b2 法条 / b3 案例 / b4 多轮, 各自一个  $\tau_b$ , 在 dev 上校准 | `lawgate/gate/calibrate.py: calibrate()` | |
| 混合门控 | `router_mode="hybrid": b2 用神经信号, b1/b3/b4 用复杂度评分 | `lawgate/router.py: RouterOptions` |  
| 生成缓存 | 内容寻址 (sha256(model\|kind|max_tokens\|prompt)), 跨方法复用 | `lawgate/cache.py: make_key()` / `GenCache` |
```

1.4 系统的"可解释性"主张

每个回答都附一个 `trace` 字典, 包含: 走哪条通道 (`channel`)、为什么 (`decision`)、门控分与阈值 (`u` / τ_b)、用的是哪种闸门 (`gate\_source`)、落在哪个桶 (`bucket`)、抽到并继承了哪些槽位 (`slots` / `slots\_inherited`)、时效状态 (`validity\_status`)、案号核验结论 (`case\_verify`)、条文官方来源 (`source\_url`)、检索命中明细 (`retrieval`)、延迟与检索次数 (`latency\_ms` / `n\_retrieval\_calls`)。

2. 运行环境与部署

2.1 实测环境 (`docs/env\_report.json`)

```
| 项 | 值 |  
|---|---|  
| 操作系统 | `Windows-11-10.0.26200-SP0` (AMD64) |  
| Python | `3.13.9` (Anaconda 打包版, `E:\Anaconda\python.exe`) |  
| 逻辑核心数 | `**18**` |  
| PyTorch | `2.11.0+cpu`, `cuda_available=false`, `cuda_device_count=0` |  
| transformers | `5.8.0` |  
| sentence-transformers | `5.5.1` |  
| chromadb | `1.5.9` |  
| gradio | `5.50.0` |  
| fastapi / uvicorn | `0.136.1` / `0.46.0` |  
| scikit-learn / scipy / statsmodels | `1.7.2` / `1.16.3` / `0.14.5` |  
| bitsandbytes | `0.49.2` (有 GPU 时用于 4bit 量化) |  
| python-docx | `1.2.0` (官方 .docx 法条导入) |  
| jieba / rank_bm25 | `0.42.1` / 已安装 |  
| numpy / pandas / matplotlib | `2.3.5` / `2.3.1` / `3.10.3` |  
| vllm | `**未安装**` (`ModuleNotFoundError`) |  
| 网络 | `huggingface.co` / `pypi.org` / `flk.npc.gov.cn` 标称可达, 但存在中间层劫持 (见 `docs/deviations.md` D0)
```

2.2 模型资源

```
| 用途 | 实际使用 | 解析顺序 |  
|---|---|---|  
| 因果语言模型 (**最终回答**, 现行默认) | **本机权重 `models/Qwen3-4B`** (= 魔搭社区的 `Qwen/Qwen3-4B`, 约 4B, 标准 transformers 架构, CPU fp16, **2026-09-13 起为默认**) | `configs/base.yaml: llm_backend: hf` + `causal_model: models/Qwen3-4B` (见 **D38**); 权重由 `scripts/download_modelscope.py` 从**魔搭 ModelScope** 拉取 (本机 HF 不可达, D0) |  
| 云端 API (可选, 更快) | **DeepSeek 官方 API 的 `deepseek-v4-flash`** (服务端回报 `model=deepseek-flash`; 地址 `https://api.deepseek.com`, **关闭思考模式**, 单条 192 token 约 **1-3 s**) | `启动服务.ps1 -LlmBackend deepseek` (或 `$env:LAWGATE_LLM_PROVIDER="deepseek"`); `configs/base.yaml: deepseek_model` (见 D30); 密钥只从环境变量 / 仓库根 `.env` 的 `DEEPSEEK_API_KEY` 读取, **代码与配置从不写明文** |  
| 降级兜底模型 | `models/fuzi-mingcha-v1_0` (夫子·明察, ChatGLM-6B 底座, 6.7B, CPU fp16, 约 1 token/s) | 由 `config.py: CAUSAL_MODEL_CANDIDATES` 的候选链自动选中 (`models/Qwen3-4B` → `models/fuzi-mingcha-v1_0` → ..., 见 D29); 也可用 `$env:LAWGATE_MODEL` 显式指定 |  
| **门控草稿模型** | **与回答模型同一份权重**: 回答走本地时 `build_draft_source()` 直接把草稿接到回答模型本身 (`answer_model` 来源, 手册 S3.5: 共享模型与前缀, prefill 不翻倍), 现行即 `models/Qwen3-4B` (**D38**); 回答切成 API 时才另加载 `base.yaml: draft_model` | `configs/base.yaml: draft_model` / `draft_source`; `LAWGATE_DRAFT_SOURCE` / `LAWGATE_DRAFT_MODEL` 可覆盖; 实际路由由 `Settings.draft_model_source` 说明 |  
| 向量编码模型 | `BAAI/bge-small-zh-v1.5` (`models/bge-small-zh-v1.5`, **512 维**, 手册指定模型) | `config.py: EMBED_MODEL_CANDIDATES` |
```

****口径提示 (D21 / D29 / D30 / D38) ****: 本机历史上先后用过 0.5B (HF 缓存快照) 与 1.5B 作回答模型, 再换成 6.7B 本地司法模型 (D29), 又切到 DeepSeek API (D30),
****现在默认回到本机权重、并换成 4B 的 `models/Qwen3-4B` (D38) ****;
 6.7B 的 fuzi-mingcha 仍在库内, 按候选链降级为兜底。

引用历史结果时必须注明其模型口径。

秒级确认“最终是谁在回答、草稿从哪来”: `E:\Anaconda\python.exe scripts\check\_backend.py`
 (按服务形态分行打印); 服务在跑时看 `GET /health` 的 `llm` 块。

可用环境变量覆盖: `LAWGATE\_MODEL`、`LAWGATE\_EMBED`、`LAWGATE\_DTYPE`、`LAWGATE\_LOCAL\_THINKING`
 (本地模型),

`LAWGATE\_LLM\_PROVIDER` (服务形态)、`LAWGATE\_DRAFT\_SOURCE`、`LAWGATE\_DRAFT\_MODEL`、
 `LAWGATE\_LLM\_THINKING`、`LAWGATE\_LLM\_TIMEOUT`、`LAWGATE\_LLM\_RETRIES`、`LAWGATE\_DOTENV` (详见
 § 2.3)。

可用 `configs/base.yaml` 覆盖: `causal\_model` / `draft\_model` / `dtype` / `local\_thinking` /
 `llm\_backend` /
 `deepseek\_model` / `draft\_source` / `device` / `load\_in\_4bit` / `k\_draft` / `signal` / `max\_new\_tokens` /
 `seeds` / `allow\_network`。

`dtype` 说明: `auto` 在 CPU 上落 fp32; ****本地权重必须显式 `float16`****

(Qwen3-4B 约 4B 参数: fp32 要约 16 GB, fp16 约 8 GB; 旧的 6.7B 兜底更极端, fp32 要约 27 GB)。

`local\_thinking` 说明 (D38): Qwen3 的 chat template 判定是

`{% if enable\_thinking is defined and enable\_thinking is false %}`——****不传这个变量 = 开思考****。

开着思考时模型先写一大段 `...`，`max\_new\_tokens=192` 很可能被 reasoning 吃光，

正文变空或截断 (与 D30 在 DeepSeek API 上的陷阱同款)。故 `base.yaml` 显式写 `local\_thinking: false`，

`HFLLM` 只在模板确实引用该变量时才传参 (`llm\_thinking\_supported` 会写进 `/health` 与溯源)。

Qwen3 是标准架构 (`AutoModelForCausalLM` + `apply\_chat\_template`)，****不走**** ChatGLM 兼容层；

ChatGLM 系 (旧版远程代码，如 fuzi-mingcha) 的加载由 `lawgate/compat\_chatglm.py` 统一处理

(中文路径的 sentencepiece、旧 tokenizer `\_pad`、`GenerationMixin`、KV cache 桥接等，见 D29)。

`allow\_network` 说明: 该开关****只约束 HuggingFace 权重下载**** (本环境对 HF 有中间层劫持，D0)，

****不是“禁止一切运行时联网”****——DeepSeek API 调用是设计内的网络行为，走不走由 `llm\_backend` 决定。

现行默认 (本机 Qwen3-4B) ****全程离线****，压根不发网络请求。

2.3 环境变量与 `.env` (D30 起 `.env` 真的被代码读取)

```
$env:KMP_DUPLICATE_LIB_OK = "TRUE" # Anaconda MKL 与 torch 冲突; 不设会崩
$env:TOKENIZERS_PARALLELISM = "false" # 关闭分词器并行
$env:HF_HUB_OFFLINE = "1" # huggingface_hub(httpx) 证书校验失败, 强制离线下载权重
```

`lawgate/\_\_init\_\_.py` 对前两项做了 `setdefault`; `lawgate/api/app.py` 与

`lawgate/api/ui.py` 在导入 gradio/torch 之前再次 `setdefault`。

上面三项是****进程级****引导, 由 `lawgate/env\_setup.py: apply()` 统一设置 (须在 import torch 前生效)。

同一个 `apply()` 会****顺带读一次仓库根的 `.env`**** (D30) ——这是本项目****唯一****读 `.env` 的地方:

变量	作用	默认 / 留空时
`DEEPSEEK_API_KEY`	DeepSeek 密钥 (**只存在于 `.env` 与环境变量中; `.env` 已 `gitignore`**)	缺失 → 调用 401, 报错写明去查 `.env`
`DEEPSEEK_BASE_URL`	API 地址	`https://api.deepseek.com`
`DEEPSEEK_MODEL`	回答模型名 (等价于 `base.yaml: deepseek_model`)	`deepseek-v4-flash`
`LAWGATE_LLM_PROVIDER`	服务形态: `deepseek` / `hf` / `rule` (**覆盖** `base.yaml: llm_backend`)	按 `llm_b`

ackend` (当前 `hf`, 即本机权重) |
 | `LAWGATE\_LLM\_THINKING` | **API 回答的**思考模式: 留空/`0` = **关** (默认, 正文非空且快); `1` = 开 | 关 |
 | `LAWGATE\_LOCAL\_THINKING` | **本机权重的**思考模式: 留空/`0` = **关** (默认); `1` = 开。 Qwen3 模板不传该开
 关 = 开思考, 会把 192 token 吃在 `` 上 (D38) | 关 |
 | `LAWGATE\_LLM\_TIMEOUT` | 单次请求超时 (秒) | `120` |
 | `LAWGATE\_LLM\_RETRIES` | 失败重试次数 (仅 429 / 5xx / 超时重试) | `3` |
 | `LAWGATE\_DRAFT\_SOURCE` | 门控草稿来源: `auto` / `local` / `api` / `none` | `auto` (回答走本地时草稿复用回答模
 型; 走 API 时 local → api → rule) |
 | `LAWGATE\_DRAFT\_MODEL` | 草稿模型路径: 设 `none` = 不加载任何本地草稿模型 | 自动挑 `models/Qwen3-4B` |
 | `LAWGATE\_DOTENV` | 设 `0` = **完全不读 `.env`** (复现“无密钥时的降级行为”) | `1` |

****优先级: 真实环境变量 > `.env`**** (已存在的变量绝不被覆盖)。因此命令行、启动脚本

`-LlmBackend` 的临时覆盖永远优先, `.env` 只是“没设时”的默认值。

`.env` **含密钥, 不要外传、不要提交**;

/health` 与 `check_backend.py` 只报“Key 有没有读到”, 从不回显明文。

2.4 部署步骤

```
cd <仓库根>
# 0) 环境变量 (同上)
# 1) 结构化事实库
python -m lawgate.knowledge.build_sqlite          # 可选 --from-raw / --report
# 2) 向量库
python -m lawgate.knowledge.build_vector          # 可选 --backend numpy / --dry-run
# 3) 自检
python scripts/quickcheck.py                      # 结果写 docs/quickcheck.txt (UTF-8)
# 4) 评测集
python scripts/build_benchmark.py --seed 42
python scripts/build_benchmark.py --verify
# 5) Web 界面 / API
python -m lawgate.api.ui --port 7860              # Gradio 双栏 + Trace
python -m lawgate.api.app --port 8000             # FastAPI (/docs 交互文档)
```

第 0 步之后可先做一次**服务形态自检** (秒级、不加载权重、不联网):

`E:\Anaconda\python.exe scripts\check\_backend.py` —— 打印“回答模型 / 地址 / Key 是否读到 / 思考模式 / 门控草稿来源”; 走 API 时还要跑 `E:\Anaconda\python.exe scripts\check\_deepseek.py` (离线段不花钱、不联网; 加 `--live` 才打真机)。****离线演示****用

`powershell -ExecutionPolicy Bypass -File .\启动服务.ps1 -LlmBackend hf` 切回本机权重 (D30、D29)。

部署优先级: 知识库 → 向量库 → 自检 → 评测集 → 服务。

`build\_vector` 依赖 `build\_sqlite` 的产物; 跳过任一步会导致对应通道退化 (见 §8)。

2.5 目录约定 (`lawgate/config.py: Settings`)

变量	默认值	说明
`db_path`	`data/kb/legal_facts.db`	结构化事实库
`chroma_path`	`data/kb/chroma`	向量库
`raw_dir`	`data/raw`	种子语料 / 官方原文落盘
`judgments_dir`	`data/judgments`	真实文书导入源 (可选)
`bench_dir`	`data/benchmark`	评测集
`results_dir` / `figures_dir` / `docs_dir` / `configs_dir`	`results` / `figures` / `docs` / `configs`	产物

生成缓存固定为 `data/kb/gen\_cache.db` (`lawgate/cache.py`)。

3. 模块说明

3.1 `lawgate/config.py` —— 全局配置与路径解析

项	说明
---	----

```

|---|---|
| 职责 | 统一路径、模型解析、运行期参数、环境指纹 |
| 关键类 | `Settings` (dataclass, 字段见上表与 §2.2) |
| 关键函数 | `get_settings(**overrides)` (进程内单例)、`Settings.provenance()`、`Settings.model_label()` |
| 私有工具 | `_load_yaml()` (无 PyYAML 时的极简兜底解析)、`_has_weights()` (**必须同时有 `config.json` 与真实权重**才视为可用)、`_first_existing_dir()`、`_snapshot_dirs()` |
| 输入 | `configs/base.yaml`、环境变量 |
| 输出 | `Settings` 实例: `provenance()` 返回硬件/设备/模型/后端/日期/版本的指纹字典, 被写入每份结果与图注 |

```

3.2 `lawgate/router.py` —— 三通道异构路由 (系统主入口)

```

| 项 | 说明 | |
|---|---|---|
| 职责 | 意图 → 通道 B → 门控 → 通道 A/C 的完整决策与 trace 输出 |
| 关键类 | `LegalGateRouter` (`__init__(llm, retriever, channel_b, taus, options, db_path)`), `RouterOptions` (dataclass) |
| 关键方法 | `answer(query, history, meta)`, `answer_many(items)`, `description()`, `_tau_for(bucket)`, `close()` |
| 关键函数 | `RouterOptions` 字段: `signal="margin"`, `k_draft=20`, `max_new_tokens=192`, `top_k=8`, `rerank=True`, `disable_channel_b=False`, `single_tau=False`, `tau_single=0.10`, `tau_scale=1.0`, `router_mode="hybrid"`, `signal_buckets=("b2",)` |
| 输入 | `query: str`, `history: list[{query, answer, trace}]`, `meta: dict` (读 `meta["temporal"]`["as_of"], `meta["bucket"]`, `meta["category"]`) |
| 输出 | `{"answer": str, "trace": dict, "provision": list|None}` |

```

****决策顺序 (与代码一一对应) ****

1. `detect\_intent(query, history, db\_path)`;
2. `b\_eligible = intent.hit\_provision or intent.hit\_case\_no or intent.slots\_complete`;
若 `b\_eligible` 且未 `disable\_channel\_b` → `ChannelB.run(intent.slots, query, as\_of)`,
返回非 `None` 则直接结束 (`channel="B"`, `n\_retrieval\_calls=0`);
3. 否则 `llm.draft\_logprobs(query, history, k=k\_draft)` → `u\_signal`;
4. `classify\_bucket(...)` 定桶 (`meta["bucket"]` 优先);
5. `complexity\_score(...)` → `u\_complexity`;
6. 按 `router\_mode` 与 `signal\_buckets` 选 `u` 与 `gate\_source`;
7. `tau\_b = taus[bucket] × tau\_scale`;
8. `u > tau\_b` → 通道 C (`n\_retrieval\_calls=1`); 否则通道 A (0);
9. 组装 `trace` 返回。

3.3 `lawgate/gate/intent.py` —— 意图检测与槽位抽取

```

| 项 | 说明 |
|---|---|
| 职责 | 全确定性意图识别 + 槽位抽取 + 槽位继承 (无模型依赖) |
| 数据类 | `Slots` (`law_short` / `article_no` / `paragraph_no` / `item_no` / `case_no_raw` / `case_no_parsed` / `topic` / `temporal_query` / `law_validity_query` / `provision_seeking` / `inherited`), `Intent` (`hit_provision` / `hit_case_no` / `slots_complete` / `route_b_reason`) |
| 关键函数 | `detect_intent()`, `extract_case_no()`, `get_dicts()`, `Dictionaries` |
| 关键常量 | `CN`, `RE_ARTICLE_REF`, `RE_CASE_NO`, `PROVISION_SEEKING`, `TEMPORAL_WORDS`, `COURT_CODE_MAP` (25 个省简称), `CASE_TYPE_MAP` (11 种类型代号), `INVALID_CASE_CHARS` ([测假编试验无效]) |
| 词典来源 | 优先 DB 表 `law_alias` (23 条) 与 `topic_keyword` (89 条), 失败退内置 `LAW_ALIAS_FALLBACK` / `TOPIC_KEYWORDS_FALLBACK` |
| 继承规则 | 逆序遍历 `history`, 只从 `turn["trace"]`["channel"] == "B" 且 `trace["slots"]` 非空的轮次取 `topic` / `law_short`; 实际继承的字段名记入 `slots.inherited` |

```

****六条 `slots\_complete` 判据 (`route\_b\_reason` 的取值) **:**

```

| `route_b_reason` | 条件 |
|---|---|

```

```
| `case_no` | 命中案号 (含“形式像案号但代字非法”的情形) |
| `law+article` | 有 `law_short` 且有 `article_no` |
| `law_validity` | 有 `law_short` 且命中时效问句 (如“担保法现在还有用吗”) |
| `topic+provision_seeking` | 有 `topic`、有法条索取标记、**且没有条号** |
| `""` (不完整) | 其它 (如纯概念题“什么是离婚冷静期”——**刻意不进通道 B**, 见 `docs/deviations.md` D10) |
```

3.4 `lawgate/gate/draft.py` —— 草稿生成

```
| 项 | 说明 |
|---|---|
| 职责 | 为门控取前 k 个 token 的 logprob 分布。**先确认它与回答模型是否同源**——现行配置下就是同一份权重 (见下) |
| 关键类 | `DraftGenerator(llm=None, model_path=None, k=20)` |
| 关键方法 | `draft_logprobs(query, history, k)`、`draft_top1()` (兼容手册签名的 `[(logprob, token), ...]`)、`describe()` |
| 输出 | `lawgate.channel.llm_base.DraftStats(logprobs, tokens, text, seconds, cached, source, attempts)` |
```

****现行默认下草稿与回答是同源；只有回答切 API 后才需要换源 (D38 / D30) ****

现行默认 `llm\_backend: hf` (回答 = 本机 `models/Qwen3-4B`) 时, 草稿**就是回答模型本身**:

`build\_draft\_source()` 见到“回答模型不是 API”就直接返回 `answer\_model` 单源链

(手册 S3.5 的原始设计: 草稿与正式生成共享同一模型与前缀, prefill 成本不翻倍)。

所以现行配置**没有“额外再加载一个草稿模型”这笔开销**, `base.yaml: draft\_model` 只是让 `/health`、图注、页脚与实际一致 (**D38**))。

回答模型换成 DeepSeek API 后 (`-LlmBackend deepseek`), “回答模型自己给 logprobs”这条路不成立:

API 的 logprobs **只在思考模式下是真实分布**, 而思考模式下拿到的只是**套路化推理前缀**

(“我们需要回答用户...”) 的分布——实测平均裕度 $top1 \quad top2 \approx **9-11 \text{ nats}**$,

算出的 $u = \exp(\text{裕度})$ 全挤在 $**0.0000-0.0012**$, 门控信号失去区分度。

因此**该模式下**草稿改由 `lawgate/channel/draft\_source.py: DraftSourceChain` 按**来源链**提供:

```
| 顺序 | 来源 (`trace.draft_source`) | 实现 | 实测 |
|---|---|---|---|
| 1 | `local` (**该模式下默认先走这条**) | `base.yaml: draft_model` (现行 = `models/Qwen3-4B`, 惰性加载, 约 8 GB) | 历史上用 0.5B 时 u 有量级差异 (0.016 / 0.026 / **0.363** / 0.079); **换 4B 后尺度会变, 须重新看分布** (D38) |
| 2 | `api` | `DeepSeekLLM.draft_logprobs()`: 强制思考模式 + logprobs, 并做**退化分布检测** (选中 token 全 0.0 或候选过半为 9999 → 抛 `DegenerateLogprobs` 换源) | u 塌缩到 0.0000-0.0012, 故不作默认 |
| 3 | `rule` | `ExtractiveLLM` 的词面构造伪分布 | **只是让链路跑通**, 不是语言模型信号 |
```

来源与每次尝试 (失败原因、耗时) 写进 trace 的 `draft\_source` / `draft\_attempts` / `draft\_seconds`

(通道 B 走查库拼装、未取草稿时记 `channel\_b` (未取草稿)),

开关为 `configs/base.yaml: draft\_source` 或 `LAWGATE\_DRAFT\_SOURCE`, 草稿模型路径为 `LAWGATE\_DRAFT\_MODEL`。

****阈值口径 (重要偏差)**:** `configs/thresholds.json` 的 τ_b 是在 ****0.5B 草稿****上校准的 (D33-3), **换成 `models/Qwen3-4B` 后尚未重新校准**。对 LegalGate 默认 hybrid 路由**无影响** (b1/b3/b4 用确定性复杂度评分, b2 的 $\tau=1.0$ 本来就不可超越), 但 ****TARG 基线 (单阈值 $\tau=0.10$) 会受影响****——**重跑 E1/E2 前必须先看 u 分布**:

```
E:\Anaconda\python.exe scripts\check_draft_u.py --out docs\check_draft_u.txt
```

```
# 走产品里那条真实草稿链, 对固定问题集打印 u / 平均裕度 / 草稿源 / 耗时, 并落 Markdown 表 (D38)
```

历史 E0-E6 结果与现行配置 (4B 回答 + 4B 草稿) **不可直接比较**。

3.5 `lawgate/gate/signal.py` —— 不确定性信号

信号名	公式	值域	语义
`margin`	$\exp(\text{mean}(\text{top1} - \text{top2}))$	$(0, 1]$	越犹豫越大
`entropy`	$\text{mean}(H(\text{top-k})) / \log(k)$	$[0, 1]$	分布越平越大
`variance`	$V/(1+V)$, $V = \text{Var}(\text{top1 logprob})$	$[0, 1]$	token 间自信度越不稳越大
`neglogp`	$N/(1+N)$, $N = \text{mean}(\text{top1 logprob})$	$[0, 1]$	平均负对数似然越大越不确定

其它导出: `SIGNALS\_CONFIDENCE` (置信度方向, 供 E0 对照)、`RAW\_SIGNALS` (未映射原始量)、
`compute\_all()`、`normalize\_signal()`、以及兼容旧签名的 `entropy\_from\_logprobs()` /
`margin\_from\_logprobs()` / `token\_entropy()`。聚合位置默认前 `m=8` 个草稿 token。

3.6 `lawgate/gate/complexity.py` —— 确定性复杂度评分

特征	权重	触发条件
`long`	+0.30	查询长度 > 30 字
`case_word`	+0.30	命中 `RE_CASE_WORD` (案例/判决/判例/类似/怎么判/如何判/判多久/裁判规则)
`topic`	+0.20	命中法律主题词
`multi_turn`	+0.10	`history` 非空
`compare`	+0.10	命中 `RE_COMPARE` (区别/能不能/是否/谁承担/哪个/更有利/冲突/例外/之外/相比/上限/下限...)
`boilerplate_penalty`	0.30 (有条号再 0.20)	命中 `RE_PROVISION_BOILER` (条号原文/的内容是什么/原文/条文/如下)
`category_prior`	+0.30 (case) / +0.20 (temporal_trap) / 0	类别先验

输出 `ComplexityBreakdown(score, features)`、`score` 截断到 `[0, 1]`、权重**人工设定、不调参**。

3.7 `lawgate/gate/calibrate.py` —— 桶级阈值校准

项	说明
职责	在 dev 上逐桶求 τ_b ，并网格搜索 TARG 的单全局 τ
桶定义	`BUCKETS = {"b1": "概念咨询", "b2": "法条查询", "b3": "案例检索", "b4": "多轮追问"}`
网格与容差	`GRID = 0.01...0.50` (50 点)、`DELTA = 0.005`
默认规则	`rule="largest_feasible"`: 满足 $\text{acc}(\tau) \geq \text{acc_AlwaysRAG} - \delta$ 的**最大** τ (即精度约束下检索代价最小的工作点)。手册原式 `smallest_feasible` 保留为对照 (`docs/deviations.md` D13)
关键函数	`calibrate()`、`calibrate_single_tau()`、`tau_scales()`、`load_taus()`、`classify_bucket()`、`bucket_descriptions()`
输入	`dev_preds: list[{bucket, u, correct_if_retrieve, correct_if_direct}]`
输出	`configs/thresholds.json` (`{"b1": $\tau$ , "b2": $\tau$ , "b3": $\tau$ , "b4": $\tau$ }`) 与逐桶报告 (含 `acc_at_tau` / `r_at_tau` / `tau_min_feasible` / `tau_max_feasible`)
无解时	取最接近目标精度的工作点并标 `feasible: false`；该桶无样本时 τ 取 0.50 并记 `reason`

****当前状态 (2026-09-13 更新, D33-3) ****: `configs/thresholds.json` ****已按新草稿源重校准****

(b1 0.29 / b2 1.0 / b3 1.0 / ****b4 1.0****, `dev\_calib` 144 条、`largest\_feasible`、`grid\_max=1.0`
见 `results/calibrate\_report.json`；2026-09-11 首校值 b4 0.49 已被取代)。

D30 的“未按新草稿重新校准”警告****已解除****；更换回答/草稿后端后仍须重走“看 u 分布 → 重校准”流程。

重跑: `python scripts/calibrate.py --split dev\_calib --grid-max 1.0` (见 `docs/DATA\_GAP.md` G8)。

`load\_taus()` 在文件缺失时返回全 0.1 (****会静默退化****, 复现时先确认该文件存在)。

3.8 `lawgate/channel/b\_structured.py` —— 通道 B 主逻辑

路径	触发条件	关键调用	trace 关键字段
P1 案号核验	`slots.case_no_parsed` 非空或 `case_no_raw` 存在	`CaseNoVerifier.verify()`	`case_verify`、`source_url`、`steps=["case_no_verify"]`
P2 法条+时效	`law_short` $\wedge$ `article_no`	`TemporalChecker.check_provision()` → `replacement_map()` / `fetch_current()`	`temporal`、`validity_status`、`replacement`、`provision_hits`、`steps=["temporal_check", "provision_render"]`

| ****P3 法律效力询问**** | `law\_short` ^ (`law\_validity\_query` 或 [时效问句且无 topic]) | `check\_law()` → 逐条
 `fetch\_current()` 渲染替代条文 | `temporal`、`validity\_status`、`replacement`、`steps=["law\_validity\_check"]` |
 | ****P4 主题条文检索**** | `topic` ^ `provision\_seeking` | FTS5 `provision\_fts MATCH ?` + `validity\_status='` 现行有效`,
 `ORDER BY rank LIMIT 8` | `topic\_hits`、`source\_url`、`steps=["topic\_fts"]` 或 `[topic\_fts\_miss->C]` |

其它要点:

- `\_fts\_query(topic)` : 把主题词做成 `“词1” OR “词2”` , 并转义 `“ \* ( ) : ^ - ` ;
- `\_render(rows, max\_rows=8)` : 渲染为 `《法名》条标 [项号] 正文` + `(来源: URL)` ;
- `\_compose\_warning(tv, rep)` : 拼装 `【现行规定】` / `【替代说明】` / `【法律沿革】` 段;
- ****返回 `None`**** 表示“查不到/槽位不全” → 由 `router` 回落到门控 (这就是“B 的降级路径”);
- 命中合成案号时, 答案末尾****自动附加**** `【数据溯源】` 本案号来自****合成案号库**** (data_source=SYNTHETIC) ...`。

3.9 `lawgate/channel/b\_temporal.py` —— 时效状态机

方法	说明
`check\_law(law\_short, as\_of)`	逐条读 `law\_lifecycle` (****仅精确匹配 `from\_law`****), 返回状态 + 警示 + 替代 + 沿革链
`check\_provision(law\_short, article\_no, as\_of)`	限定 `paragraph\_no=1 AND item\_no=''` , 取 `ORDER BY version DESC LIMIT 1` ; 先判法级状态, 再判条级 `validity\_status` , 最后判 `effective\_date > as\_of`
`replacement\_map(law\_short, article\_no=None)`	先用库内 `superseded\_by` (形如 `民法典#667` , 正则 `([`#`]+)#(\d+)`), 再用种子 `SUPERSEDE\_MAP`
`fetch\_current(law\_short, article\_no)`	按 `version DESC, paragraph\_no, item\_idx` 返回全部款项行
`article\_text(law\_short, article\_no, version)`	把“条”的全部款项拼回完整正文
`\_trace\_chain(src, max\_depth=6)`	收集****全部后继****、按生效日排序、去重, 形成完整沿革链

状态取值: `现行有效` / `已修订` / `已废止` / `尚未生效` / `查无此条`。

3.10 `lawgate/channel/b\_case\_verify.py` —— 案号核验器

项	说明	
级别常量	`LEVELS = ["格式非法", "不存在", "存在但类型不符", "存在但案由不符", "核验通过"]`	
关键类/方法	`CaseNoVerifier.verify(parsed, claimed\_cause)`、`verify\_raw(raw, claimed\_cause)`、`log(result, expected)`; `VerifyResult.to\_dict()`	
年份合理性	`year\_range=(1990, 当前年+1)` (`\_\_init\_\_` 参数可改)	
案由匹配	`\_cause\_match()` : 去掉“纠纷”后****双向包含**** (“借贷” “民间借贷纠纷” 视为一致)	
判决顺序	① 格式 (能否解析 / 非法代字 / 年份越界 / 法院代字不以省级简称开头) → ② 存在性 (查 `case\_registry.exists\_in\_db`) → ③ 类型一致性 → ④ 案由一致性	
输入	`parsed: dict` (`extract\_case\_no()` 的产物, 或 `{}` 表示“格式像但非法”)、`claimed\_cause: str	None`
输出	`VerifyResult(level, passed, detail, real\_case, parsed\_detail)`	

3.11 `lawgate/channel/c\_semantic.py` —— 通道 C 检索

项	说明
关键类	`Retriever(col\_path, backend, embedder, reranker)`; `RetrievalResult`
关键方法	`search(query, top\_k, rerank)` (返回拼接好的上下文文本)、`search\_hits(...)` (返回 `list[Hit]`)、`retrieve(...)` (返回 `RetrievalResult`)、`count()`
检索流程	`encode\_queries([query])` → `store.query(qv, top\_k=cand\_k)` , 其中 `cand\_k = max(top\_k, top\_k\*3)` (`rerank=True` 时) → `reranker.rerank(query, hits, top\_n=top\_k)`
上下文拼装	法条: `[法名条标] 正文` ; 文书: `[案号 案由] 正文` ; `max\_chars=300` 截断
线程安全	`self.\_lock` 串行化检索与编码器调用 (检索约 50 ms, 相对生成可忽略)
惰性加载	`store` / `embedder` 为 `@property` , 首次访问才加载

3.12 `lawgate/channel/llm\_base.py` + `deepseek\_llm.py` —— LLM 后端与降级链

| 后端 | `backend` 值 | 说明 |
 |---|---|
 | `DeepSeekLLM` (****默认****, `lawgate/channel/deepseek\_llm.py`, D30) | `deepseek` | DeepSeek 官方 API (OpenAI 兼

容`/chat/completions`)。整段生成、流式 (SSE)、草稿 logprobs (**强制思考模式** + 退化分布检测)。默认`thinking=False`：该模型默认开思考,`max\_tokens`小时 token 会被 reasoning 吃光、`content`为**空字符串** (实测`max\_tokens=32`→全是 reasoning_tokens), 故必须关掉。错误分级为人话 (401/402/429/5xx) 并指数退避重试,`call\_log`记逐次用量与命中缓存与否,`describe()`给出地址/模型/思考开关/累计 token |

|`HFLLM`|`hf`| transformers + 本地权重; **现为离线兜底** (`models/fuzi-mingcha-v1\_0`, D29); 精度由`Settings.dtype`决定 (本机`float16`); 有 CUDA 时尝试`BitsAndBytesConfig(load\_in\_4bit=True, bnb\_4bit\_compute\_dtype=torch.bfloat16)` , 异常则退 fp16 + `device\_map="cuda:0"`。 **ChatGLM 系** (`config.json: model\_type=chatglm`, 如 fuzi-mingcha) 走`lawgate/compat\_chatglm.py`的兼容装载 (`llm\_backend\_impl=chatglm-compat`) , 提示词用原生`[Round n]问/答`模板、tokenizer 自动补`[gMASK] <sop>`、KV cache 由桥接壳自管 |

|`VLLMLLM`|`vllm`| 部署后端,`LLM(model=..., enable\_prefix\_caching=True)`; 接口与`HFLLM`一致 (**本机未安装 vllm**) |

|`ExtractiveLLM`|`rule\_based`| 无权重时的确定性抽取式兜底;`draft\_logprobs()`用词面构造伪 logprob, **不是**语言模型信号 |

共同接口: `generate(query, history, context, max\_tokens)`、`generate\_stream(query, history, context, max\_tokens)`、

`draft\_logprobs(query, history, k)`、

`build\_prompt(...)`、`describe()`。`get\_llm(prefer="auto"|"deepseek"|"hf"|"vllm"|"rule")`按下面的顺序探测可用性。

提示词模板: `SYSTEM\_BASE` (法律咨询助手 + 不得编造法条或案号) 与

`SYSTEM\_WITH\_CONTEXT` (附加`【参考资料】`段);`PromptMixin.build\_messages()`拼装

system + history + 当前问题,`HFLLM.build\_prompt()`优先用

`tok.apply\_chat\_template(..., add\_generation\_prompt=True)` , 失败退纯文本拼接。

`DeepSeekLLM`没有 tokenizer (API 直接吃`messages`数组), 因此缓存键里的`prompt`是

messages 的 JSON 序列化 ; 人设类基线改走`generate\_messages()` (见 §8.2 的 D30-3c) 。

降级链 (D38 起顺序回到“本地优先”) : `prefer="auto"`时先看`Settings.llm\_provider / llm\_backend` :

(llm_backend: hf, 当前默认) HFLLM ——失败—— VLLMLLM ——失败—— ExtractiveLLM (规则兜底)
(llm_backend: deepseek) DeepSeekLLM (**直接返回, 不降级到本地**, 理由见 §8.2)

显式`get\_llm("deepseek")`时**不静默换模型**——失败直接把异常抛出去 (否则会出现“改了配置却跑出

另一套结果”) ; 本地链每次降级的原因都`print`到 stdout, 详见 §8.2。

流式输出 (D28-2) : `generate\_stream()`返回增量文本生成器。

`HFLLM`用`TextIteratorStreamer` + 后台线程真·逐 token (`model.generate(streamer=...)`)

是阻塞调用, 不放线程就无法边算边推; 线程里的异常搬回主线程重抛);

`VLLMLLM`用离线引擎`add\_request` + `step()`循环 (本机无 vllm, 未实测);

`BaseLLM`默认实现与`ExtractiveLLM`整段产出一段 (规则兜底没有 token 概念)。

流式与整段**共用同一把缓存键与同一套贪心参数** , 因此结果一致、缓存互通。

耗时构成与 GPU 估算 (`scripts/\_timing\_breakdown.py`, 实测见`docs/\_timing\_breakdown.txt` ;

下表取自**本地权重后端** , 用于回答“换 GPU 能快多少”) :

本机一“问” (192 token, CPU-only) 合计约 69 s, 其中**神经计算占 95.4%**

(门控草稿 20 token = 4.9 s、预填充到首字 = 10.1 s、解码 192 token = 51.0 s @ 3.76 tok/s),

检索每问仅 0.03 s (冷启动加载才慢)。故换 GPU 的收益≈生成速度倍数:

消费级 GPU + vLLM 下整问约 2-7 s (约 10-30×), 首字 0.3-1 s;

`transformers`单流路径在 GPU 上仍偏慢, 详见 README §1「一“问”到底慢在哪」。

现行默认 (本机`models/Qwen3-4B`, fp16/CPU, D38) 的耗时构成 : 门控草稿与回答模型是

同一份权重 , 因此**没有“单独的草稿耗时”这一段**——整条延迟 = 一次 prefill + 逐 token 解码,

分段实测见`docs/deviations.md` D38; 检索仍是毫秒级; 命中内容缓存 ≈0.1 s。

>

****D30 的 API 模式****(`-LlmBackend deepseek`)：生成只要约 ****1 s****，“首字”延迟主要由 ****取门控草稿**** 决定（那段历史里草稿是 0.5B，约 1.5–3 s；实测 3.3 s 里 2.4 s 是草稿），命中内容缓存整体 ≈ 0.05 s。

3.13 `lawgate/cache.py` —— 生成缓存

项	说明
关键类/函数	`GenCache(path="data/kb/gen_cache.db", enabled=True)`、`make_key(model, kind, max_tokens, prompt)`、`get_cache()`
key	`sha256(f"{model}\x00{kind}\x00{max_tokens}\x00" + prompt)`
表	`gen_cache(key, kind, model, max_tokens, payload, n_tokens, seconds, created)`、`gen_stats(key, hits, misses)`
并发	`threading.local()` 连接 + `threading.Lock()` 写锁 + `PRAGMA journal_mode=WAL`
用途	折叠跨方法重复生成 (NeverRAG/ComplexityRouter 直答/TARG 直答/LegalGate 通道 A 共享同一 prompt)
当前规模	**只读查询 `data/kb/gen_cache.db`** (`gen_cache` 行数 / `gen_stats` 命中与 miss)；2026-09-13 实测 **5429 条** (deepseek-v4-flash generate 3188 条 ≈ 100.9 万 token，见 `docs/check_deepseek_cache.txt` / 台账)，早期台账的“275 条”已随 D33 全量实验过期

3.14 `lawgate/knowledge/\*` —— 知识库

文件	职责	关键函数/类
`schema.sql`	手册 schema 主体 (含 D1/D2 修正)	建表见 §5
`schema_ext.sql`	溯源与质量扩展表	`ingest_provenance` / `parse_audit` / `case_verify_log`
`flk_parser.py`	法条解析状态机	`parse_law_text()` / `parse_law_file()` / `render_article()` / `validate_continuity()` / `cn2int()` / `int2cn()` / `normalize_lines()` / `Provision`
`audit.py`	解析质量审计	`audit_law()` / `sample_for_manual_audit()` / `LawAudit.roundtrip_rate`
`seed_corpus.py`	13 部法律的关键条文种子语料 + 沿革 + 替代映射 + 词典	`LAW_SPECS` / `LIFECYCLE_SEED` / `SU PERSEDE_MAP` / `LAW_ALIAS_SEED` / `TOPIC_KEYWORDS_SEED` / `SEED_VERSION="seed-2026.09"`
`seed_cases.py`	合成案号与合成文书生成	`generate_cases()` / `render_full_text()` / `COURTS` / `CAUSE_ACTIONS` / `CASE_TYPES` / `FACT_TEMPLATES`
`judgment_parser.py`	案号抽取/解析/入库	`RE_CASE_NO` / `extract_case_no()` / `parse_case_no()` / `case_no_fields()` / `build_registry()` / `manual_import()`
`build_sqlite.py`	建库主流程	`build()` / `create_schema()` / `ingest_laws()` / `seed_lifecycle()` / `seed_dictionaries()` / `build_keywords()` / `build_registry()` / `write_qa_report()` / `write_manual_audit_csv()`
`build_vector.py`	向量库构建	`build()` / `collect_records()` / `chunk(size=300, overlap=50)`
`store.py`	向量后端	`ChromaStore` / `NumpyStore` / `Hit` / `get_store(path, collection, prefer)`
`embed.py`	文本编码	`BGEEmbedder` / `HashingCharEmbedder` / `get_embedder()` / `BGE_QUERY_INSTRUCTION`
`rerank.py`	重排	`Reranker.rerank()` ($0.60 \cdot \text{BM25归一} + 0.25 \cdot \text{字符覆盖率} + 0.15 \cdot \text{结构命中}$)、`tokenize()`
`risk_terms.py`	判分词表	`ABOLISH_MARKERS` / `REFUSAL_MARKERS` / `UNCERTAIN_MARKERS` / `STOPWORDS`

3.15 `lawgate/eval/\*` —— 评测

文件	职责	关键函数/类
`benchmark.py`	评测集构建	`build_benchmark()` / `verify_benchmark()`；常量 `CASE_DATA_IS_SYNTHETIC=True`、`FROZEN_AS_OF="2026-11-01"`、`N_*`、`BUCKET`、`DEV_RATIO=0.20`、`SPLIT_SEED=42`、`CONCEPT_NEED_RETRIEVAL_RATIO=0.40`
`metrics.py`	机械代理判分与汇总	`score_item()` / `aggregate()` / `case_verify_prf()` / `extract_citations()` / `key_term_recall()` / `invalid_law_citations()` / `ItemMetrics`
`baselines.py`	6 个统一接口方法	`NeverRAG` / `AlwaysRAG` / `TARG` / `ComplexityRouter` / `LegalLLMPersona` (代理基线) / `LegalGate`；`build_methods()` / `count_tokens()`
`run_exp.py`	实验框架	`RunContext` / `build_context()` / `run_method()` / `run_experiment()` / `merge_shards()` / `shard_items()` / `load_invalid_laws()`
`io.py`	I/O 与图注	`load_split()` / `load_results()` / `write_jsonl()` (**拒绝静默覆盖**) / `write_summary_csv()` / `stamp_footer()` / `run_meta()`
`stats.py`	统计检验	`paired_bootstrap(n=10000)` / `wilcoxon()` / `cohens_d()` / `holm_bonferroni()` / `compare()`

```
| `plotting.py` | 出图 | `setup_cjk_font()` / `provenance_footer()` / `plot_e0_distributions()` / `plot_pareto()`  
` / `plot_ablation()` / `plot_multiturn_trend()` / `plot_tvc_by_trap()` / `plot_confusion()` / `plot_latency_budget`  
`);` DEMO_MARK` |
```

3.16 `lawgate/api/\*` —— 服务与界面

```
| 文件 | 职责 | 关键对象 |  
|---|---|---|  
| `app.py` | FastAPI 服务 | `app` (title `CA-LegalGate 律核`, version `0.1.0`)、`get_router()` (惰性单例)、模型  
`ChatReq` / `CaseReq` / `TemporalReq` / `Turn` |  
| `ui.py` | Gradio 双栏界面 | `build_demo()` / `respond_stream()` (流式生成器) / `build_trace_html()` / `ensure`  
`loaded()` / `gen_max_tokens()` / `column_max_tokens()` / `real_case_no()` / `PRESETS` / `STATE` |
```

4. 三个典型使用场景

场景 1：查已废止法条（杀手用例）

****用户输入****：「《合同法》第52条规定哪些情形合同无效？」

****处理链****：

```
1. `detect_intent` → `slots={law_short:"合同法", article_no:52, topic:"合同效力", provision`  
`_seeking:true}` ,  
`slots_complete=true`, `route_b_reason="law+article"`;  
2. 通道 B **P2**: `TemporalChecker.check_provision("合同法", 52)`  
→ `_lifecycle_rows()` 精确匹配命中 `合同法` → 民法典` 的废止记录  
→ `status="已废止"`、`is_safe=False`;  
3. 因 `not tv.is_safe`, 调 `replacement_map("合同法", 52)` 取 `SUPERSEDE_MAP` 的  
`民法典#153`, 渲染 `_compose_warning()`;  
4. trace: `validity_status="已废止"`、`replacement=[...]`、`source_url=None`、  
`must_show_warning=true`、`steps=["temporal_check"]`。
```

****实测返回****（`docs/quickcheck.txt` §4）：

```
《合同法》第52条已随该法已废止。| 原文条（1999）：有下列情形之一的，合同无效：… |  
《合同法》已于 2021-01-01 被废止，现行规定见《民法典》。… |  
【现行规定】《民法典》第一百五十三条：违反法律、行政法规的强制性规定的民事法律行为无效，… |  
（来源：https://flk.npc.gov.cn/detail2.html?…） | 【替代说明】[第52条→]…
```

****对照****：左栏“无门控直答”通常仍照引已废止的《合同法》第 52 条——这就是系统的价值点。

****相关****：时效状态机的判定边界已由 `docs/quickcheck.txt` §2 覆盖：

```
`公司法47 @2024-06-30` = 尚未生效`、`@2024-07-02` = 现行有效`、  
`公司法(2018修正)26` = 已修订`、`婚姻法32` = 已废止`、`民法典第9999条` = 查无此条`。
```

场景 2：核验案号

****用户输入****：「（2019）鲁01民终1242号 这个案子什么案由？」

****处理链****：

```
1. `extract_case_no()` 用 `RE_CASE_NO` 解析出  
`{year:2019, court_code:"鲁01", court_region:"山东", case_type:"民终", case_type_name:"民事二`  
`审", seq_no:1242, normalized:"（2019）鲁01民终1242号"}`;  
2. `slots_complete=true`, `route_b_reason="case_no"`;  
3. 通道 B **P1**: `CaseNoVerifier.verify(parsed, slots.topic)`  
→ ① 格式通过 → ② `case_registry` 命中且 `exists_in_db=1` → ③ 类型一致 → ④ 案由一致 →
```

`核验通过`;

4. trace: `case\_verify={level, passed, detail, real\_case, parsed}`、`source\_url`、
`must\_show\_warning=false`、`steps=["case\_no\_verify"]`。

****实测返回**** (`docs/quickcheck.txt` §3):

案号 (2019)鲁01民终1242号 核验通过: 山东省济南市中级人民法院, 案由「民间借贷纠纷」, 民事二审。

【文书信息】山东省济南市中级人民法院 | 案由: 民间借贷纠纷 | 裁判日期: 2019-04-05

【数据溯源】本案号来自****合成案号库**** (data_source=SYNTHETIC), 用于系统联调, 不对应真实案件, 不得作为真实引用依据。

四级判定的验收用例全部通过 (`docs/quickcheck.txt` §3, 6/6):

`核验通过` / `存在但案由不符` / `格式非法 (年份越界 2099)` / `不存在` /
`格式非法 (缺括号)` / `格式非法 (非法类型代字"测")`。

结论口径: 案号库为****合成****数据, "不存在"的判定只对本库成立;

真实场景中"库里没有但确实存在"的****假阴性****无法在本库上测量** (`docs/deviations.md` D15)。

场景 3: 多轮追问 (槽位继承)

****对话****:

- T1: 「我朋友借我10万不还, 适用哪条法律?」

→ `slots={topic:"民间借贷", provision\_seeking:true}` → `route\_b\_reason="topic+provision\_seeking"`

→ 通道 B ****P4****: `provision\_fts MATCH "借钱" OR "借款" OR ...` 且 `validity\_status='现行有效'`

→ trace 的 `slots.topic="民间借贷"` 被写入 `history[-1].trace`;

- T2: 「那我能主张多少利息?」

→ T2 自身****没有****主题词, 但 `detect\_intent` 逆序扫描 history 发现上一轮

`trace.channel == "B"` 且 `trace.slots.topic == "民间借贷"`

→ `slots.topic` 被继承, `slots.inherited = ["topic"]`;

→ 若 T2 命中法条索取标记则继续走通道 B, 否则进入门控 (`classify\_bucket` 因 `history` 非空判为 ****b4****);

- 门控: `bucket="b4"`, `router\_mode="hybrid"` 且 `b4` signal_buckets`

→ `gate\_source="complexity"`, `u = complexity\_score(...)` (含 `multi\_turn` +0.10);

- 评测时 `metrics.score\_item()` 会比对基准题的 `slots.inherited` 与 trace 的
`slots\_inherited`, 得到 `slots\_inherited\_ok` (多轮槽位继承正确率)。

****评测集侧对应****: `data/benchmark/` 的 100 组多轮题 (每组 T1/T2/T3, 共 300 条),

T2 明确带 `slots.inherited=["topic", "cause\_action"]`; E3 用 `test\_e4` (20 组 = 60 条)

画每轮 acc - RR 趋势与槽位继承错误率。

5. 数据结构说明

5.1 结构化事实库 `data/kb/legal\_facts.db`

****当前实测规模**** (`data/kb/qa\_report.md` 库构建报告 + `data/kb/legal\_facts.db` 只读 SQL 探测):

表	行数	备注
--- --- ---		
`legal_provisions`	**158**	去重 (law_short, article_no) = **95** ; 现行有效 **80** ; 13 部法律; 状态分布: 现行有效 131 / 已废止 26 / 已修订 1
`provision_fts`	158	FTS5 虚拟表 (含 4 张影子表)
`provision_keywords`	100	关键词 → (法, 条) 反向索引

`law\_lifecycle`	8	法律沿革条目
`law\_alias`	23	法律别名
`topic\_keyword`	89	主题词
`case\_registry`	**600**	`data\_source='SYNTHETIC'` 600/600
`judgments`	**600**	`data\_source='SYNTHETIC'` 600/600
`ingest\_provenance`	13	全部 `source\_kind=MANUAL\_TRANSCRIPT`、`verification=PENDING\_FLK\_VERIFICATION`
`parse\_audit`	0	人工抽验记录表（待人工填写）
`case\_verify\_log`	0	核验日志表（`CaseNoVerifier.log()` 写入）

`legal\_provisions`（法条主表，款/项级）

列	类型	说明
`provision\_id`	INTEGER PK AUTOINCREMENT	主键；FTS 精确回连用（D1）
`law\_name`	TEXT NOT NULL	《中华人民共和国民法典》
`law\_short`	TEXT NOT NULL	民法典
`law\_level`	TEXT	法律/行政法规/司法解释
`book` / `chapter` / `section`	TEXT	编 / 章 / 节
`article\_no`	INTEGER NOT NULL	数字条号
`article\_label`	TEXT NOT NULL	第六百六十七条
`paragraph\_no`	INTEGER DEFAULT 1	款号
`item\_no`	TEXT DEFAULT ``	``（一）`；**默认空串而非 NULL**（D2，令 UNIQUE 约束真正生效）
`item\_idx`	INTEGER DEFAULT 0	项序整数索引（D7，按 `(paragraph\_no, item\_idx)` 排序）
`text`	TEXT NOT NULL	正文
`validity\_status`	TEXT NOT NULL CHECK	现行有效 / 已修订 / 已废止 / 尚未生效 / 部分失效
`effective\_date` / `publish\_date` / `version`	TEXT	生效日 / 公布日 / 版本号
`superseded\_by`	TEXT	形如 `民法典#667`
`amendment\_note`	TEXT	修订说明
`source\_db`	TEXT DEFAULT `FLK`	来源库标识
`source\_url`	TEXT NOT NULL	官方来源 URL
`retrieval\_date`	TEXT NOT NULL	获取日期
约束	`UNIQUE(law\_short, version, article\_no, paragraph\_no, item\_no)`	
索引	`idx\_lp\_query` / `idx\_lp\_status` / `idx\_lp\_ver`	

`provision\_fts`（FTS5 全文索引）

CREATE VIRTUAL TABLE provision_fts USING fts5(
text, law_short, article_no, provision_id UNINDEXED, tokenize='unicode61');

`unicode61` 对中文**按字切分**，故通道 B 的主题查询直接用主题词字面 OR (`\_fts\_query()`)。

`provision\_keywords`

`(keyword, law\_short, article\_no, weight REAL DEFAULT 1.0)`，索引 `idx\_pk\_kw(keyword)`。

`case\_registry`（案号真值库）

列	说明
`case\_no`	TEXT **PRIMARY KEY**，形如 `(2022)沪01民终12345号`
`year` / `court\_code` / `court\_name`	年份 / 法院代字 / 法院全名
`case\_type` / `seq\_no`	案件类型（如“民事二审”） / 序号
`cause\_action`	案由（民间借贷纠纷 / 劳动争议 / 离婚纠纷 / 房屋租赁合同纠纷）
`judgment\_date`	裁判日期
`exists\_in\_db`	INTEGER NOT NULL DEFAULT 1
`source\_url` / `doc\_hash`	来源 URL / 文书哈希
`data\_source`	**SYNTHETIC**（当前全部）** / `CJWS`（真实导入后）
索引	`idx\_cr\_cause(cause\_action)` / `idx\_cr\_court(court\_code)`

`judgments`

`judgment\_id` PK、`case\_no` REFERENCES case_registry(case_no)、`court\_name`、`cause\_action`、
`full\_text`、`chunks`（JSON 数组字符串，300 字 / 50 字重叠）、`data\_source`；索引 `idx\_j\_case`

。

`law\_lifecycle` (沿革表)

`from\_law`、`to\_law`、`relation` (CHECK: 废止/修订/替代/修正/新法优于旧法)、
`effective\_date`、`authority`、`note`; 索引 `idx\_ll\_from`。当前 8 条。

`law\_alias` / `topic\_keyword` (确定性词典)

`law\_alias(alias PK, law\_short);` `topic\_keyword(topic, keyword, PRIMARY KEY(topic, keyword))`

。

`detect\_intent` 与 `ChannelB.\_load\_topic\_keywords()` 优先读这两张表, 读不到退内置常量。

`ingest\_provenance` (溯源扩展表)

`law\_short` + `version` 为主键; `source\_kind` (FLK_DOCX / FLK_WEB / OFFICIAL_WEB /
MANUAL_TRANSCRIPT / SEED)、`source\_url`、`retrieval\_date`、
`verification` (**VERIFIED** / **PENDING_FLK_VERIFICATION** / UNVERIFIED)、
`verified\_by`、`verified\_date`、`n\_provisions`、`continuity\_gaps` (JSON 数组字符串)、`note`。

`parse\_audit` / `case\_verify\_log`

`parse\_audit(audit\_id, law\_short, article\_no, paragraph\_no, field, verdict CHECK('ok'|'mismatch'), auditor, audit\_date, note)`

`case\_verify\_log(log\_id, case\_no, claimed\_cause, expected, got\_level, got\_passed, run\_date)`

5.2 向量库 `data/kb/chroma/`

项	值 (`data/kb/vector_build.json`)
--- ---	
后端	`chromadb` (`ChromaStore`, 集合 `legal`, `hnsw:space=cosine`; 失败自动退 `NumpyStore`)
编码后端	`bge-small-zh-v1.5`
维度	**512**
文档数	**1295** = 法条 **95** + 文书分块 **1200** (来自 **600** 篇合成文书)
构建耗时	37.8 s (CPU)

文档 ID 规则: 法条 `lp\_{law\_short}\_{version}\_{article\_no}`; 文书块 `jd\_{case\_no}\_{i}`。

元数据: 法条含 `type/law/article/article\_label/validity\_status/version/url`;

文书含 `type=judgment/case\_no/cause/url`。

5.3 评测集 `data/benchmark/`

文件	内容
--- ---	
`all.jsonl`	1180 条 (**唯一权威全集**)
`dev.jsonl` / `test.jsonl`	236 / 944 (分层 20/80, `split_seed=42`)
`dev_calib.jsonl` / `test_e1.jsonl` / `test_e4.jsonl`	144 / 300 / 60 (分层子集)
`temporal_trap.jsonl` / `case_no_verify.jsonl` / `test_multiturn.jsonl`	专题划分
`counts.json`	条数口径、构建参数、`kb_stats`、`data_caveats`
`construction_log.md`	逐类生成方式与全部披露
`verify_report.md`	自校验报告 (结论 PASS)
`kappa_report.md`	κ 报告 (**当前为模拟标注员**)
`subsets_manifest.json`	子集构成与抽样理由
`simulated_annotators/`	模拟投票文件 (仅流水线自检, 不得作为证据)

****单条条目的字段模板**** (`benchmark.py: ITEM\_KEYS`, 所有键****始终存在****且****只允许****这些键):

```
qid, query, history, category, bucket, need_retrieval, golden_answer,
golden_provisions, golden_source, slots, temporal, case_no, turn_id, group_id,
annotators, arbitrated, gold_pass, split
```

`temporal` 子键: `as\_of` / `expect\_status` / `trap\_type` / `superseded\_by`;
`case\_no` 子键: `raw` / `exists` / `true\_cause`。

5.4 结果文件 `results/`

文件	格式
`results/<exp>/<method>_seed<k>_<split>.jsonl`	逐条记录 (`run_exp.run_method()` 产出, 字段见下)
`results/<exp>/summary_<method>_seed<k>_<split>.json`	方法级汇总 (`metrics.aggregate()`)
`results/<exp>/shards/..._part<i>.jsonl`	分片结果 (`nshards>1` 时)
`results/<exp>/summary.csv`	E1 汇总表 (`io.write_summary_csv()`)

逐条记录字段: `qid, category, bucket, method, seed, split, turn_id, group_id, gold_pass, correct, retrieval_used, n_retrieval_calls, latency_ms, n_tokens, u, tau_b, tau, channel, gate_source, router_mode, tv, case_pass, invalid_law_cited, invalid_laws, cited_golden, key_recall, refusal, uncertain, answer_len, slots_inherited_ok, score_detail, answer, trace`。

方法级汇总字段: `n, acc, rr, arc, lac, tv, tv\_n, case\_verify\_acc, case\_verify\_n, invalid\_law\_citation\_rate, slot\_inherit\_acc, slot\_inherit\_n, p50\_ms, p95\_ms, mean\_ms, mean\_tokens, total\_tokens, n\_refusal` (外加 `method/seed/split/exp/tag/shard/elapsed\_s`)。

****命名说明****: `acc` = 准确率; `rr` = 检索调用率; `arc` = 平均检索调用数;
`lac` 是 `arc` 的手册命名别名 (**不含延迟维度**, 延迟由 `p50\_ms` / `p95\_ms` / `mean\_ms` 单独报告);
`tv` = 时效性正确率。

5.5 生成缓存 `data/kb/gen\_cache.db`

见 § 3.13。该文件本身可作为“没有偷跑”的审计证据: 它记录了每条 prompt 的哈希、token 数与耗时。

6. API 说明

启动:

```
python -m lawgate.api.app --host 127.0.0.1 --port 8000  
# 或 uvicorn lawgate.api.app:app --host 127.0.0.1 --port 8000
```

`ROUTER` 为**进程内单例、惰性初始化** (`get\_router()`), 首次请求时才加载模型与向量库。

6.1 `GET /health`

健康检查 + 后端溯源。返回:

```
{  
  "status": "ok",  
  "uptime_s": 12.3,  
  "provenance": {"hardware": "...", "device": "cpu", "causal_model": "Qwen3-4B",  
    "causal_model_path": "models/Qwen3-4B", "causal_model_dtype": "float16",  
    "embed_model": "bge-small-zh-v1.5",  
    "llm_provider": "local", "llm_api_base": null,  
    "llm_api_model": null, "llm_thinking": null, "has_api_key": false,  
    "draft_source": "auto", "draft_model": "Qwen3-4B",  
    "draft_model_path": "models/Qwen3-4B",  
    "llm_backend": "hf", "date": "...", "version": "0.1.0"},  
  "router_loaded": true,  
  "load_error": null,  
  "llm": {"provider": "local", "backend": "hf", "model": "Qwen3-4B",  
    "api_base": null, "thinking": null, "key_present": false,  
    "draft_source": "auto", "draft_model": "Qwen3-4B",  
    "note": "本字段只反映配置, 不发起任何请求; 走 API 时 (-LlmBackend deepseek) 连通性用 scripts/check_deepseek.py 验。"},  
}
```

```

    "cache": {"enabled": true, "by_kind": {...}, "total_entries": 275,
              "total_tokens": 8377, "total_compute_seconds": 2148.6}
  }

```

`llm` 块是 D30 新增的: 服务形态 (本地 / API) 此前只能靠翻配置猜, 现在一眼可查——

`provider` / `backend` / `model` / `api\_base` / `thinking` / `key\_present` /
`draft\_source` / `draft\_model`。它**只反映配置, 不发起任何请求** (因此不花钱、不受断网影响);
真连通性用 `scripts/check\_deepseek.py --live` 验。`key\_present` 只报“有没有读到 Key”, **从不回显明文**。

6.2 `POST /chat`

请求体 (`ChatReq`):

```

{"query": "《合同法》第52条规定哪些情形合同无效?",
 "history": [{"query": "上一轮问题", "answer": "上一轮回答"}],
 "as_of": "2026-11-01"}

```

- `history[].trace` 若携带 (如 Gradio 侧会带上), 可用于**槽位继承**;
- `as\_of` 会包装为 `meta["temporal"]["as\_of"]`, 供时效状态机使用评估时点。

响应:

```

{"answer": ".....", "trace": { ... 见 §6.5... }, "provision": [{...}] | null}

```

6.2b `POST /chat/stream` —— **流式**问答 (SSE, D28-2)

请求体与 `POST /chat` **完全相同** (`ChatReq`)。响应是 `text/event-stream`,
逐帧推送 (每帧 `event: <名>` + `data: <JSON>`, UTF-8):

```

event: stage
data: {"stage": "channel_b" | "retrieval" | "generate", "channel": "A", "u": 0.42, "tau_b": 0.29, "bucket": "b1",
      "decision": "..."}

event: delta
data: {"text": "根据"}

event: done
data: {"answer": "..... (权威全文)", "trace": { ... 同 §6.5... }, "provision": null}

event: error
data: {"message": "RuntimeError: ..."}

```

约定与边界:

项	说明
事件顺序	`stage` * → `delta` + → `done`; 生成期异常以 `error` 收尾 (不会静默断流)
权威全文	客户端必须以 `done.answer` 收口; `delta` 只是增量 (逐片解码在极少数“多字节字符被切成两个 token”时会短暂出现差异)
通道 B	走的是查库拼装, **没有 token 流**, 整段一次性 `delta` (不假装逐字打字)
与 `/chat` 的关系	路由决策、trace 字段**完全一致** (两者消费 router 的同一份 `_answer_events`), 只有生成方式不同
事件循环	端点是同步 `def` + `StreamingResponse`, 由 Starlette 线程池逐帧取数, CPU 阻塞不占事件循环

命令行看流式 (PowerShell):

```

curl.exe -N -X POST http://127.0.0.1:8010/chat/stream `
-H "Content-Type: application/json" `
-d '{"query": "《合同法》第52条规定哪些情形合同无效?"}'

```

6.3 `POST /verify\_case`

请求体 (`CaseReq`): `{ "case\_no": " (2019) 鲁01民终1242号", "claimed\_cause": "劳动争议" }`

响应即 `VerifyResult.to\_dict()`:

{ "level": "存在但案由不符", "passed": false, "detail": " 案号 ... 真实存在, 但真实案由为「民间借贷纠纷」, 与所述「劳动争议」不符。...",

"real_case": { "case_no": "...", "court_name": "...", "cause_action": "...",
"case_type": "...", "judgment_date": "...", "data_source": "SYNTHETIC"},
"parsed": { "year": 2019, "court_code": "鲁01", "case_type": "民终",
"case_type_name": "民事二审", "seq_no": 1242 }}

该接口直接构造 `CaseNoVerifier()` 并调用 `extract\_case\_no()` ,
不经 router, 因此**不消耗**任何模型算力 (毫秒级)。

6.4 `POST /temporal`

请求体 (`TemporalReq`): `{ "law\_short": "公司法", "article\_no": 47, "as\_of": "2024-06-30" }`

- `article\_no` 给定 → `TemporalChecker.check\_provision()`; 否则 → `check\_law()`;
- 响应为 `TemporalVerdict.to\_dict()`, **并附加** `replacement\_map` 字段:

{ "status": "尚未生效", "is_safe": false, "warning": "该条将于 2024-07-01 生效, 截至 2024-06-30 尚未施行",
"replacement": null, "chain": ["公司法"], "as_of": "2024-06-30", "matched_law": "公司法",
"replacement_map": [{ "law_short": "民法典", "article_no": 153, "note": "..." }] }

6.5 `GET /trace/schema`

返回 trace 字段的中文说明 (答辩演示用)。**实际返回的字段清单**如下表 (与 `router.answer()` 一致):

字段	含义
channel	B = 确定性结构化 / A = 直接生成 / C = 语义检索生成
u	门控决策分 ([0,1] , 越大越倾向检索)
u_signal	草稿 logprob 的不确定性信号值
u_complexity	确定性复杂度评分
draft_source	**本次 u 的草稿来自谁 (D30)**: `api` (回答模型的 API logprobs) / `local` (本机小模型) / `rule` (确定性伪分布) / `channel_b` (未取草稿)
draft_attempts	**换源过程**: 每一次尝试的来源、耗时、成功与否及失败原因
draft_seconds	取草稿耗时 (秒) —— API 后端下它才是“首字延迟”的主要成分
gate_source	neural_signal \ complexity —— 本桶实际采用的闸门
router_mode	hybrid \ signal \ complexity
complexity_features	七维特征明细 (`long` / `case_word` / `topic` / `multi_turn` / `compare` / `boilerplate_penalty` / `category_prior`)
tau_b	该桶的校准阈值 (`taus[bucket] × tau_scale`)
bucket	b1 概念 / b2 法条 / b3 案例 / b4 多轮
slots	抽取到的槽位 (含继承结果)
slots_inherited	实际继承的槽位名列表
validity_status	时效状态 (现行有效 / 已修订 / 已废止 / 尚未生效 / 查无此条)
case_verify	案号核验四级判定结果 (`level` / `passed` / `detail` / `real_case` / `parsed`)
source_url	条文官方来源
decision	人类可读判定串, 如 [complexity] u=0.4000 > tau_b2=0.1 → C
latency_ms	端到端延迟 (毫秒)
n_retrieval_calls	本次检索调用次数 (0/1)
signal	当前配置的信号名 (`margin` / `entropy` / `variance` / `neglogp`)
k_draft	草稿长度
intent	意图检测完整结果 (`hit_provision` / `hit_case_no` / `slots_complete` / `route_b_reason` / `slots`)
retrieval	检索命中明细: `top_k` / `reranked` / `seconds` / `backend` / `embed_backend` / `hits` [{ doc_id, score, type, law, article, case_no }]
llm_backend	实际生效的 LLM 后端 (`deepseek` / `hf` / `vllm` / `rule_based`; 草稿走来源链时另见 `draft_source`)
draft_text	草稿前 200 字 (可人工核对闸门判断是否合理)
steps	通道 B 额外字段 (如 [`temporal_check`, `provision_render`])、 `temporal`、 `replacement`、 `provision_hits`、 `topic_hits`、 `must_show_warning`

| 对照字段 | 通道 B 返回时 `u` / `tau\_b` 置 `null`（未走门控） |

7. 界面说明

启动：`python -m lawgate.api.ui --port 7860`（可选 `--host`、`--share`）。

7.1 布局

```
## 律核 LawGate • 双栏对照演示

### 左栏：无门控直答（对照）
gr.Markdown()
← STATE["base_llm"]
.generate_stream(...)
恒不检索

### 右栏：律核（可解释路由）
gr.Markdown()
← router.answer_stream(...)
+ 【结构化命中】段（若有 provision）

gr.Textbox(label="你的法律咨询", lines=3)
gr.Textbox(label="评估时点（可选，YYYY-MM-DD；留空=今天）"） [提问]

#### 演示预设（6 个按钮，点击即填入输入框）

### Trace 面板（每一步决策可解释、可复核、有官方来源）
gr.HTML() ← build_trace_html(trace)（流式期间先显示阶段提示）
gr.Markdown() ← 备注行（左/右栏通道、**两栏统一上限**、右栏耗时）
gr.State([]) ← history（每轮追加 {query, answer, trace}）
```

7.2 流式输出与两栏口径（D28）

- * 处理函数 `respond\_stream()` 是**生成器**（Gradio 流式 = 每次 `yield` 刷新一次组件）；左栏在后台线程写 `queue`、右栏在本线程按 `router.answer\_stream()` 的事件推进，两栏**同时**长出来，而不是一栏等另一栏。
- * **两栏生成长度上限必须一致**（否则“对照”不公平）：
- * 唯一来源 `gen\_max\_tokens()` = `Settings.max\_new\_tokens`，现默认 **192**（一键脚本的 `~UiMaxTokens` → `LAWGATE\_UI\_MAXTOK`；`configs/base.yaml: max\_new\_tokens`）；
- * `ensure\_loaded()` 用 `RouterOptions(max\_new\_tokens=mt)` 构造 router
- 右栏（通道 A/C）的生成参数取自 router options（与 `eval/run\_exp.py` 的评测约定一致）；
- * `respond\_stream()` 再以 **router options 为准**把同一个数显式传给左栏
- 两处不会各自漂移（历史缺陷：左栏 80、右栏 192，见 `docs/deviations.md` D28-3）；
- * 想临时缩短演示等待：`启动服务.ps1 -UiMaxTokens 80`（两栏会一起变短，口径仍一致）。
- * **接口侧同口径**：`/chat`、`/chat/stream` 用 `RouterOptions` 默认 192 token，与界面统一到同一个数——不再存在“界面一套、接口另一套”。

7.2 演示预设按钮（`ui.py: PRESETS`）

| 按钮 | 预置问题 | 演示重点 |

---|---|---

| ① 已废止法律（杀手用例） | 「《合同法》第52条规定哪些情形合同无效？」 | 左栏照引废止法 vs 右栏废止警示 + 民法典替代条 + 沿革链 |

| ② 编造案号 | 「（2099）沪01民终88888号 这个案子是什么案由？」 | 右栏“格式非法（年份越界）” |

| ③ 真实案由 + 错误案由 | 运行时由 `real\_case\_no()` 从 `case\_registry` 取一条 `民间借贷纠纷` 案号并拼接 | 右栏“存在但案由不符” |

| ④ 法律是否仍有效 | 「担保法现在还有用吗？」 | 走 P3 路径返回废止结论 + 沿革链 |

| ⑤ 纯概念题 | 「什么是离婚冷静期？」 | **不进通道 B** (D10 修正点) |

| ⑥ 多轮追问 | 「那我能主张多少利息？」 | 槽位继承与 Trace 面板中的 `继承槽位` |

7.3 Trace 面板显示的字段 (`build\_trace\_html()`) 的行顺序)

```
| 行 | 取值 |
|---|---|
| 通道 | `trace["channel"]` |
| 门控来源 | `trace["gate_source"]` |
| 决策 | `trace["decision"]` |
| u (决策分) | `trace["u"]` |
| u_signal (神经信号) | `trace["u_signal"]` |
| u_complexity (复杂度) | `trace["u_complexity"]` |
|  $\tau_b$  | `trace["tau_b"]` |
| 桶 | `trace["bucket"]` |
| 槽位 | `json.dumps(trace["slots"])` |
| 继承槽位 | `trace["slots_inherited"]` |
| **草稿来源 (D30) ** | `trace["draft_source"]` (本次 u 是哪个草稿源给的) |
| **草稿耗时 (ms) (D30) ** | `trace["draft_seconds"]` ( $\times 1000$  取整; 无此字段时该行留空) |
| 时效状态 | `trace["temporal"]["status"]` 或 `trace["validity_status"]` |
| 案号核验 | `trace["case_verify"]["level"]` |
| 来源 | `trace["source_url"]` |
| 延迟 (ms) | `trace["latency_ms"]` |
| 检索调用 | `trace["n_retrieval_calls"]` |
```

此外:

- `trace["must\_show\_warning"]` 为真时, 面板顶部显示橙色警示条
「 本条命中时效/核验风险, 必须向用户显式提示」;
- `trace["retrieval"]["hits"]` 非空时, 折叠面板列出前 8 条命中的
`score / type / law / article / case`;
- 底部固定显示「通道 B 步骤: `trace["steps"]`」与界面左下角的
「左栏=无门控直答 右栏=律核 (通道) 右栏耗时 N ms」。

7.4 界面页眉与页脚声明 (按**实际生效**的后端动态措辞)

页眉 (`ui.py` 的 `build\_demo()`) 与页脚都从 `Settings.provenance()` **动态生成**, 不再写死模型名

(历史上写死过一次并把 0.5B/1.5B 口径搞错过, 见 D21、D29; D30 换 API 后又一次证明必须动态取)

。

```
| 场景 | 页眉措辞 (要点) | 页脚声明 (要点) |
|---|---|---|
| **本地模式 (现行默认) ** | CPU 上首字几秒到几十秒出现, 整段受生成长度上限约束 (命中内容缓存  $\approx 0.1$  s); 语言模型在本机 CPU 上运行; 门控草稿与回答是**同一份权重** | 「本项目运行在 `` 环境, 语言模型为 **`Qwen3-4B`** (`<dtype>` 精度, 本机权重), 单栏最多生成 192 token, 向量模型为 bge-small-zh-v1.5; 法条语料为人工录入的关键条文 (PENDING_FLK_VERIFICATION), 案号库为合成数据。」 |
| **API 模式** (`-LlmBackend deepseek`) | 每次提问约 **1-3 秒**出答案 (DeepSeek API, 关闭思考模式), 命中内容缓存后  $\approx 0.05$  s; 语言模型走 DeepSeek 官方 API, **不需要**在本机加载本地权重; 门控草稿用本机草稿模型 (现行 `Qwen3-4B`), **首次提问**会多花时间加载它 | 同上, 但语言模型为 **deepseek-v4-flash** (DeepSeek 官方 API: `https://api.deepseek.com`; 思考模式 关) |
```

页眉页脚一律由 `Settings.provenance()` **动态取** (`lawgate/api/ui.py` 的 `build\_demo()`) , 所以上表只是“措辞要点”, 实际文字会跟着 `configs/base.yaml` 走——这是 D21/D29/D30/D38 反复出现“文档写死模型名 → 与配置不符”之后定下的做法。

页脚另有一行**草稿来源声明** (D30/D38) :

「门控草稿来源 `auto` (本机 `` → API logprobs → 确定性评分; 回答模型走本地时草稿直接复用回答模型本身, 手册 S3.5; 每条 trace 的 **草稿来源** 行会写明本次实际用了哪条)」。

8. 错误处理与降级路径

系统按“**绝不静默失败、绝不混淆降级结果与正式结果**”的原则设计。全部降级都会
在 `trace` 或产物文件中留下可核对的标记。

8.1 通道内降级

层	触发条件	处置	留痕
通道 B	槽位不全 / 查无此条 / 主题 FTS 未命中	返回 `None` → 回落门控	trace `steps` 记 `miss->C` / `topic_fts_miss->C`; `validity_status`="查无此条"
通道 B	法条命中但法/条已失效	不返回条文, 改返回废止警示 + 替代条文	`must_show_warning=true`、`replacement=[...]`
通道 B	法条时效查询抛 `ValueError` (日期解析失败)	跳过该沿革行继续匹配	`check_law()` 内 `try/except` (TypeError, ValueError)
通道 B	FTS 查询语法错误	捕获 `sqlite3.Error`, 按“未命中”处理	`steps` 记 `topic_fts_miss->C`
通道 B	`case_verify_log` 表缺失	`CaseNoVerifier.log()` 静默跳过 (**日志失败不得影响核验主流程**)	无痕 (设计如此)
通道 C	chromadb 不可用	`get_store()` 自动退 `NumpyStore` (精确余弦, 走 `embeddings.npy` + `records.jsonl`)	`RetrievalResult.backend` = `numpy`
通道 C	编码模型不可用	`get_embedder()` 退 `HashingCharEmbedder` (字符 n-gram 哈希, 不是语义模型)	`RetrievalResult.embed_backend` = `HashingCharEmbedder(dim=512, ngram=1-2-3)`
通道 C	BM25 依赖缺失或语料为空	`Reranker._bm25()` 返回全 0, 仅用字符覆盖 + 结构命中	`Reranker.name` 为 `char+struct`
生成	缓存命中	直接返回, `cache.hit_miss()` 记 hit	记录 `cached=true` (草稿)
生成	API 返回 401/402/429/5xx、超时、连接被重置	`DeepSeekLLM` 分级成人话报错 (提示查 `.env` 的 `DEEPSEEK_API_KEY`), 429/5xx/超时按 `LAWGATE_LLM_RETRIES` 指数退避重试	`DeepSeekError` 逐层冒泡到界面/接口; `call_log` 记逐次用量与错误数
门控	草稿源不可用 (本机小模型缺权重 / API logprobs 退化 / 无任何模型)	`DraftSourceChain` 按 local → api → rule 换源, 绝不“用假分布算出一个看起来很正常的 u”	trace `draft_source` + `draft_attempts` (哪条失败、原因、耗时)

8.2 模型层降级链 (`get\_llm()`)

现行默认 (D38): `configs/base.yaml` 的 `llm\_backend` 是 `hf`, 即**直接走本机权重** (`models/Qwen3-4B`, 不联网、不需要任何云端凭据); 加载失败才逐级降级到 vLLM / 规则。

场景	行为
`llm_backend: hf` (**当前默认**)	直接走本机权重 `models/Qwen3-4B` (完全离线); 加载失败才退 vLLM / 规则兜底
`llm_backend: deepseek`	DeepSeekLLM (直接返回, 不自动降级 ——理由见下)

解析结果若为 `deepseek` 就**只返回 `DeepSeekLLM`**: `DEEPSEEK\_API\_KEY` 缺失只 `print` 一条警告, 真正的问题在**调用时**以 401 暴露 (人话报错会提示去查 `.env`)。**刻意不静默降级到本地模型**——否则“改了配置却跑出另一套结果” (`docs/deviations.md` D30 的教训)。想要离线就显式 `--llm-backend hf`

- 失败原因会被 `print` 到 stdout (形如 `[llm] HF 后端加载失败 (...)`)、
走 DeepSeek API 但没有读到 DEEPSEEK_API_KEY (.env 或环境变量), 调用会以 401 失败`
- 兜底后端的产出必须被标注**: `trace["llm\_backend"]` 会写 `rule\_based`, 使用者据此把该批结果与神经模型结果分开;
- 不是语言模型**: 有 context 时按词面重叠选句子, 无 context 时

直接说明“未提供参考资料，无法给出可靠结论”，`draft\_logprobs()` 返回的是词面构造的伪 logprob。

8.3 服务层

场景	行为
首次请求时模型/向量库加载失败	`get\_router()` 抛异常并把 `type(exc).\_\_name\_\_` 记入 `\_LOAD\_ERROR`；`health` 返回 `router\_loaded=false` 与该错误串
`/chat` 单条处理异常	`run\_exp.run\_method()` 侧捕获为 `{ "answer": "[ERROR] ...", "trace": { "channel": "error", "n\_retrieval\_calls": 0 } }`；界面侧显示 `[右栏失败] 类型: 消息`
Gradio 未加载成功	左右两栏都显示 `\*\*加载失败\*\*`：`<错误串>`，Trace 面板显示 `trace 不可用`
输入为空	返回空字符串与 `「请输入问题」`
结果文件已存在	`io.write\_jsonl()` 抛 `FileExistsError`（**拒绝静默覆盖**），需显式 `--overwrite`
图谱数据缺失	`plotting.\_no\_data()` 画一张 `NO DATA` 占位图，**不抛异常**，不中断整批出图
流水线单步失败	`run\_pipeline.py` 每步 `try/except`，失败原因写入 `docs/pipeline\_steps.json`，后续步骤继续执行（避免“部分失败被当成全部成功”）

8.4 数据层“可见的降级”（最重要的一条）

凡涉及**非最终正式数据**的产物，都在文件内自带声明，绝不依赖口头说明：

降级	声明位置
合成案号/文书	`case\_registry.data\_source='SYNTHETIC'`；通道 B 答案尾部追加 【数据溯源】段；`counts.json` 的 `data\_caveats`；`results/e6/prf.json` 的 `threshold\_note`
人工录入法条	`ingest\_provenance.verifcation='PENDING\_FLK\_VERIFICATION'`；`data/raw/sources.json` 逐法 `note`；`data/kb/qa\_report.md` § 3
规则化标签	`counts.json` 的 `need\_retrieval\_source`；每条 `slots.need\_retrieval\_source='rule-based'`；`construction\_log.md` § 6
模拟标注员 κ	`kappa\_report.md` 标题即警告 + 正文“结论口径（必须照抄进论文，禁止改写）”
演示图	`figures/demo/DEMO\_README.json` 的 `warning`；每张图的图注 `DEMO DATA / NOT REAL RESULTS`（`plotting.DEMO\_MARK`）
抽样子集	`subsets\_manifest.json` 的 `coverage\_note`（历史口径注记：D19/D24 时代 E1 曾仅覆盖 test 的 31.8%；*D33 起 E1 主对比已用 test 全量 944**，`test\_e1` 300 现为 E2 消融口径）

9. 性能与算力管理

9.1 实测性能

配置	单条延迟 (p50)	来源
现行默认：本机 `models/Qwen3-4B`（约 4B，fp16/CPU），`max\_new\_tokens=192`	分段与总耗时见 `docs/deviation.s.md` D38；草稿与回答同一份权重，**无独立草稿段**；缓存命中 $\approx 0.1\text{ s}$	`docs/check\_qwen3\_4b.txt`（D38）
DeepSeek API 后端，`max\_new\_tokens=192`（`-LlmBackend deepseek`，D30）	**1-3 s**（缓存命中 $\approx 0.05\text{ s}$ ；“首字” $\approx$ 取草稿 1.5-3 s）	`docs/deviations.md` D30、`docs/check\_deepseek\_live.txt`（整段 1.07 s / 57 字）
本地权重后端，`max\_new\_tokens=128`	18.9 s	`results/timing/summary\_alwaysrag\_seed0\_test.json`（p50 18877.6 ms）
本地权重后端，legalgate，`max\_new\_tokens=128`	15.6 s	`results/pilot/summary\_legalgate\_seed0\_test.json`（p50 15599.1 ms）
4 进程 × 3 线程，`max\_new\_tokens=128`	**163.8 s**（mean 155.5 s）	`results/timing/summary\_alwaysrag\_seed0\_test\_part1.json`
本地 6.7B 兜底模型，192 token 单栏	约 **2-4 分钟**（约 1 token/s）	`docs/deviations.md` D29（`docs/fuzi\_e2e.txt` 走 `--max-tokens 48`）
历史吞吐指纹（0.5B，20 token 生成）	1.62 s，**12.36 tok/s**	原载 `docs/smoke\_s0.json`（该文件与 `scripts/smoke\_s0.py` 已于 2026-09-16 删除，D37；数字保留为历史记录）
子集预算下的实测口径	80-token 生成约 7-8 s/条	`data/benchmark/subsets\_manifest.json` 的 `why`

上表除第一行外都是**本地权重后端**的读数（历史口径）。**现行默认**就是本地权重

（`models/Qwen3-4B`，D38）：整条延迟 = 一次 prefill + 逐 token 解码，无单独的草稿耗时；若切到 API（`-LlmBackend deepseek`），“单条延迟”主要由网络往返决定，

本机 CPU 只影响门控草稿 (那一路用的是本地草稿模型) 与向量检索 (毫秒级)。

****结论: 线程级并发是负收益。**** torch 的 intra-op 并行已吃满内存带宽, 再叠加 inter-op 并发只会互相抢核; 同时 `ChannelB` (SQLite) 与 chromadb 客户端****都不是线程安全的**** (`RunContext` 因此按线程各建一份 `ChannelB`/`router`, 并对 `Retriever` 加锁串行)。
正确做法是****多进程分片**** (`--shard i --nshards n`, 由用户开多个终端启动), 而不是提高 `workers`。

9.2 三项算力管理机制

机制	实现	效果
内容寻址生成缓存	`lawgate/cache.py`	折叠跨方法重复 prompt; 当前 275 条缓存覆盖 8377 token / 2148.6 s 的实算量
分层子集	`scripts/make\_subsets.py` → `subsets\_manifest.json`	E1 从 944 条降到 300 条, ****但保留全部时效陷阱与案号核验****
分片并行 + 合并	`--shard/--nshards` + `run\_exp.merge\_shards()`	用多个独立进程跑; 分片结果落 `results/<exp>/shards/`, 合并后写正式结果文件
其它可调项: `max\_new\_tokens` (CPU 下建议 80, GPU 下可 192 - 256)、`top\_k` (默认 8)、`rerank` (默认开)、`k\_draft` (默认 20; 消融 A4 覆盖 8/16/20/32/64)。

9.3 图与结果的溯源要求

`lawgate/eval/io.py: stamp\_footer()` 为每张图生成图注:
`硬件 | 模型 | 向量模型 | 后端 | 日期 | 版本 [ | 附加说明]`。
`lawgate/eval/plotting.py: provenance\_footer()` 统一渲染, 并支持附加
`DEMO DATA / NOT REAL RESULTS` 等额外行。****每张图都必须带该图注****, 否则视为不合格产物。

10. 已知限制与免责声明

- **本系统的输出不构成法律意见****, 不得用于真实法律意见、诉讼文书引用或任何法律决策依据。
详见 `docs/model\_card.md` 的“超出范围的使用”。
- **法条语料为人工录入的关键条文, 不是官方原文****: `legal\_provisions` 158 行 / 95 个去重条文, 全部 `MANUAL\_TRANSCRIPT` + `PENDING\_FLK\_VERIFICATION`; ****《民法典》种子语料只有 62 条**** (非全量 1260 条)。导入官方原文的路径见 `docs/DATA\_GAP.md` G1。
- **案号库与文书全部为合成数据**** (`data\_source='SYNTHETIC'`, 600 + 600 条), 不对应真实案件; E6 的满分是****构造性****结果 (评测集子类与核验器判定级别同源), 不能外推为对真实裁判文书网的准确率。
- **标签为规则化生成****, 本环境无人工标注员; κ 报告来自模拟标注员。
- **最终回答默认由本机权重生成**** (`models/Qwen3-4B` = 魔搭 `Qwen/Qwen3-4B`, 约 4B, fp16/CPU, D38):
由此带来三条必须披露的限制——(a) ****没有 GPU 的代价****: CPU 逐 token 解码决定 192 token 的等待量级 (实测见 `docs/deviations.md` D38); 想快就切云端 API, 代价是联网 + 按 token 计费;
(b) ****门控草稿与回答是同一份权重**** (手册 S3.5: 共享模型与前缀, prefill 不翻倍), 因此没有“单独的草稿耗时”这一段, 也没有 6.7B 时代那 13 GB 的额外常驻;
(c) ****绝对准确率仍不可与手册目标值对比**** (手册指定 7B, 且法条语料非官方原文)。
****可选加速路径****: `~LlmBackend deepseek` 走 DeepSeek 官方 API (`deepseek-v4-flash`, 关思考, 单条 192 token 约 1 - 3 s, D30), 该模式下 (a) ****联网依赖与计费****: 每问一次 HTTPS 请求 (命中内容缓存则 0 请求), 断网/无密钥会明确报错并提示查 `.env` 的 `DEEPSEEK\_API\_KEY`;
(b) “首字”延迟主要由****取门控草稿****决定, 不是生成本身。

****降级兜底****为本地 6.7B 司法模型 (`fuzi-mingcha-v1\_0`, ChatGLM 底座, CPU ****fp16****, D29):

(a) fp16 复现精度、非 fp32/bfloat16; (b) 贪心解码下长句****偶发重复打转****;

(c) 约 ****1 token/s**** (192 token 单栏 2-4 分钟), 故离线演示常配 `-UiMaxTokens` 或用缓存。

方法间对比因共用同一模型与超参而仍然公平。

历史结果 (pilot/timing/E0 为 0.5B、E1 为 1.5B、E1-E5 为“API 回答 + 0.5B 草稿”、

D29 为本地 6.7B) 口径各不相同, 引用须注明 (D21/D29/D30/D33/D38)。

6. ****桶级阈值已按新草稿源重校准 (D33-3, 2026-09-12)****: `configs/thresholds.json` 现为

b1 0.29 / b2 1.0 / b3 1.0 / b4 1.0 (`dev\_calib` 144 条, 见 `results/calibrate\_report.json`;

b1 仍 `feasible:false`, 0.29 为 `largest\_feasible` 兜底值)——换草稿/回答后端会改变 u 的分布尺度,

重跑前仍须先看 u 分布再重校准。E1-E5 已在新口径下全部跑完 (D33/D34)。

7. ****自动判分是机械代理判据****, 不是人工判定; 对“答案表述质量”只能给近似分。

8. 逐条偏差 (D0-D35) 与影响见 `docs/deviations.md`; 风险与处置见 `docs/risk\_register.md`; 验收判定见 `docs/ACCEPTANCE.md`。